

DOCUMENTAZIONE DI PROGETTO

SocialApp — Build Week 3

Architettura, storia ed esempi di codice del progetto e di tutto il lavoro svolto

Aggiornato il 23 luglio 2026 · React 19 + Redux Toolkit + Firebase (Authentication + Firestore + Storage)

Indice

1. Cos'è il progetto
2. Come è costruito il progetto
3. Flusso dell'applicazione
4. Le fasi precedenti alla migrazione a Firebase (storia, backend JSON Server)
5. Passaggi fatti nelle sessioni di lavoro (1–14, incluse migrazione Firebase e rifiniture mobile)
6. File principali toccati
7. Stato attuale

1. Cos'è il progetto

SocialApp è un social network didattico (esercizio "Build Week 3", corso Epicode): autenticazione, feed con post/commenti/reazioni, profilo utente con follower, messaggistica privata in tempo reale, notifiche e ricerca. Il frontend è React 19 + Redux Toolkit; il backend è **Firebase** — Authentication (email/password), Firestore (database documentale, con listener `onSnapshot` per il realtime) e Storage (solo per i video allegati ai post). Fino al 22 luglio 2026 il backend era invece un mock JSON Server + JSON Server Auth (JWT, permessi Unix rwx per collection) esteso con Socket.IO per il realtime — la sezione 4 di questo documento ne racconta la storia, la sezione 5 (passaggio 12) racconta la migrazione stessa e le sue motivazioni.

2. Come è costruito il progetto

Struttura delle cartelle

<code>firestore.rules</code>	regole di sicurezza Firestore (sostituiscono <code>server/routes.json</code>)
<code>storage.rules</code>	regole di sicurezza Storage (solo video dei post)
<code>firebase.json</code> , <code>firestore.indexes.json</code>	configurazione progetto Firebase
<code>scripts/</code>	
<code>demoData.json</code>	dati demo sorgente (sostituisce <code>server/db.seed.json</code>)
<code>seedFirestore.js</code>	script una tantum (Firebase Admin SDK): crea utenti demo in Firebase Auth e popola Firestore
<code>src/</code>	
<code>main.jsx</code>	bootstrap tecnico: crea il root DOM, monta <code><App /></code>
<code>App.jsx</code>	radice logica: Redux Provider, BrowserRouter, sottoscrizione a <code>onAuthStateChanged</code> + hook realtime (<code>presence/attività/conversazioni/notifiche</code>)
<code>firebase.js</code>	inizializza l'app Firebase, esporta <code>auth/db/storage</code>
<code>app/</code>	
<code>store.js</code>	<code>configureStore</code> : un reducer per feature + middleware
<code>toastMiddleware.js</code> , <code>loadingBarMiddleware.js</code>	
<code>api/</code>	un file per risorsa, chiamate dirette all'SDK Firestore/Auth/Storage
<code>features/</code>	uno slice Redux per dominio (<code>auth</code> , <code>posts</code> , <code>comments</code> , <code>messages</code> , <code>users</code> , <code>notifications</code> , <code>search</code> , <code>follow</code> , <code>presence</code>) – <code>createAsyncThunk</code> + <code>extraReducers</code>
<code>components/</code>	componenti "di presentazione", divisi per dominio (<code>posts/</code> , <code>comments/</code> , <code>messages/</code> , <code>profile/</code> , <code>layout/</code> , <code>common/</code> , ...)
<code>pages/</code>	componenti "contenitore": leggono lo store, orchestrano i componenti di presentazione
<code>hooks/</code>	hook riusabili: <code>useActivityRealtime</code> , <code>useConversationsRealtime</code> , <code>useNotificationsRealtime</code> , <code>usePresence</code> , <code>useDebounce</code> , <code>useTheme</code>
<code>routes/</code>	<code>AppRouter</code> , <code>ProtectedRoute</code> , <code>PublicRoute</code>
<code>utils/</code>	funzioni pure: validazioni, <code>date</code> , <code>toast</code> , <code>confirm</code> , <code>link preview</code>

Pattern service + slice (ripetuto identico per ogni dominio)

Ogni risorsa ha un file in `src/api/` che parla solo con l'SDK Firestore (nessuna logica Redux), e uno slice in `src/features/` che lo usa dentro `createAsyncThunk`. La paginazione non è più "numero di pagina" (compatibile con `_page / _limit` di json-server) ma a **cursor**: un cursore-snapshot di Firestore non è serializzabile nello stato Redux, quindi si usa il valore ISO di `createdAt` dell'ultimo elemento caricato. Esempio reale, `src/api/postsService.js`:

```
export async function fetchPosts({ cursor, limit = 5, userIds } = {}) {
  if (userIds && userIds.length === 0) return { posts: [], hasMore: false, nextCursor: null }
  const constraints = [orderBy('createdAt', 'desc')]
  // Firestore 'in' accetta al massimo 10 valori per query
  if (userIds) constraints.push(where('userId', 'in', userIds.slice(0, 10)))
  if (cursor) constraints.push(where('createdAt', '<', cursor))
  constraints.push(fbLimit(limit))

  const snapshot = await getDocs(query(collection(db, POSTS), ...constraints))
  const posts = snapshot.docs.map(toPost)
  return { posts, hasMore: posts.length === limit, nextCursor: posts.length ? posts[posts.length - 1].createdAt : null }
}
```

Nello slice, ogni azione asincrona segue sempre la stessa forma — `pending` imposta lo stato di caricamento, `fulfilled` aggiorna i dati, `rejected` salva l'errore, invariato rispetto a prima della migrazione: cambia solo cosa fa il service sotto, non la forma dello slice.

Autenticazione: Firebase Auth + `onAuthStateChanged`

Non c'è più un token JWT salvato a mano in `localStorage` né un interceptor Axios: Firebase Auth gestisce da solo sessione e refresh del token. Il profilo esteso (username, bio, avatar, esperienze...) vive nel documento `users/{'uid'}` su Firestore, non nel token — va recuperato a parte dopo il login. Un solo listener, montato una volta in `App.jsx`, sostituisce sia il vecchio ripristino sessione sia il vecchio evento globale `auth:expired` (`src/api/authService.js`):

```
export function subscribeToAuthChanges(callback) {
  return onAuthStateChanged(auth, async (firebaseUser) => {
    if (!firebaseUser) { callback(null); return }
    const user = await fetchUserProfile(firebaseUser.uid) // profilo esteso da Firestore
    callback(user)
  })
}
```

Permessi: Firestore Security Rules

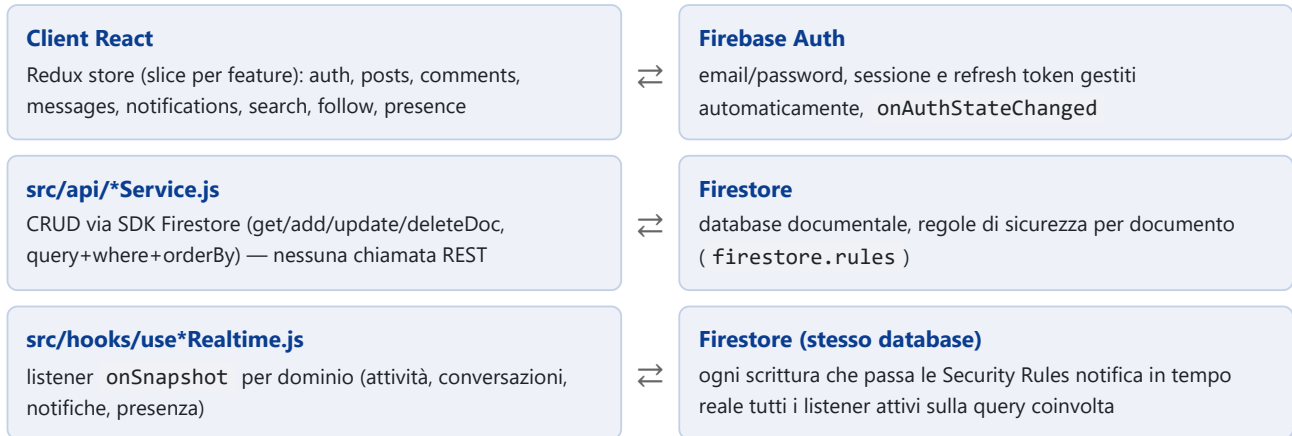
Al posto di `server/routes.json` (codici ottali Unix 640/660 per collection, ownership riconosciuta solo su un singolo campo), `firestore.rules` valuta l'autorizzazione per singolo documento e può verificare la proprietà su più campi contemporaneamente — risolve il vecchio limite "autorizzazione solo client-side" per messaggi/conversazioni (due partecipanti, non un singolo `userId`):

```
match /messages/{messageId} {
  // participantIds copiato dalla conversazione sul messaggio stesso: una query
  // "list"/onSnapshot viene negata in blocco se il campo controllato dalla
  // regola non coincide con quello del filtro lato client, anche senza get().
  allow read: if isSignedIn() && request.auth.uid in resource.data.participantIds;
  allow create: if isSignedIn()
    && request.resource.data.userId == request.auth.uid
    && request.auth.uid in request.resource.data.participantIds
    &&
  isParticipant(get(/databases/{database}/documents/conversations/{(request.resource.data.conversationId)}).data);
  allow update: if isSignedIn() && request.auth.uid in resource.data.participantIds
    && (isOwner(resource.data.userId) ||
  request.resource.data.diff(resource.data).affectedKeys().hasOnly(['read']));
  allow delete: if isOwner(resource.data.userId);
}
```

`posts / comments / likes / follows / notifications` hanno regole più semplici modellate sullo stesso schema (ownership su `userId / actorId`).

3. Flusso dell'applicazione

Il client parla con tre servizi Firebase in parallelo, tutti raggiunti direttamente dall'SDK client (nessun backend proprio in mezzo):



In pratica: quando un utente scrive un post, commenta, reagisce o inizia a seguire un altro utente, il thunk Redux corrispondente scrive direttamente su Firestore tramite il service; chiunque abbia un listener `onSnapshot` attivo su quella collection (tutti gli utenti loggati, per attività pubbliche come post/commenti/like/follow) riceve l'aggiornamento in tempo reale senza bisogno di un WebSocket o di un server intermedio — è Firestore stesso a notificare i client sottoscritti. Messaggi e conversazioni usano invece listener filtrati (`participantIds array-contains uid`), perché sono privati a due partecipanti. Le notifiche sono create **client-side** da chi compie l'azione (niente Cloud Function, per restare sul piano gratuito Spark): le Security Rules impediscono di crearne una per sé stessi o spacciandosi per un `actorId` diverso dal proprio uid, ma non c'è validazione server-side reale — limite noto, documentato in README.

`src/hooks/useConversationsRealtime.js` — un ramo del pattern (schema ripetuto per ogni dominio):

```
const unsubscribeMessages = onSnapshot(
  query(collection(db, 'messages'), where('participantIds', 'array-contains', currentUser.id)),
  (snapshot) => {
    snapshot.docChanges().forEach((change) => {
      const message = { id: change.doc.id, ...change.doc.data() }
      if (change.type === 'added') {
        dispatch(messageReceived(message))
        if (message.userId !== currentUser.id) dispatch(unreadCountIncremented(message.conversationId))
      }
      if (change.type === 'modified') dispatch(messageUpdatedFromSocket(message))
      if (change.type === 'removed') dispatch(messageDeletedFromSocket({ id: message.id, conversationId: message.conversationId }))
    })
  },
)
```

La presenza online/offline non ha un equivalente di `onDisconnect` di Realtime Database: ogni client scrive un heartbeat (`presence/{uid}.lastActiveAt`) ogni 25s, "online" è definito come "aggiornato negli ultimi 45s", ricalcolato lato client. Limite noto: una disconnessione brusca (crash, rete che cade) lascia l'utente "online" fino al timeout, non è istantanea come con un contatore di connessioni lato server.

4. Le fasi precedenti alla migrazione a Firebase

Questa sezione descrive lo stato del progetto **prima** della migrazione a Firebase del 22 luglio 2026 (backend JSON Server + JSON Server Auth + Socket.IO): è storia, non l'architettura attuale — vedi sezioni 1–3 per come funziona oggi, e il passaggio 12 più sotto per il racconto della migrazione stessa.

Fase 2 (milestone M0-M9) — l'implementazione di base

Scaffold React + Vite adattato (il progetto era già stato inizializzato con React 19/Vite 8/oxlint prima di iniziare, invece di React 18/ESLint come originariamente ipotizzato in fase di design — si è scelto di adattarsi allo scaffold esistente). Backend mock JSON Server + JSON Server Auth con dati demo, store Redux con uno slice per dominio (auth/posts/comments/users/messages), autenticazione completa, layout e navbar, profilo utente, feed con CRUD di post/commenti/like, messaggistica privata con CRUD completo, stati di caricamento/errore/vuoto su ogni schermata, autorizzazioni lato client. Verificato con un test end-to-end reale in browser (Playwright headless).

Due decisioni di questa fase restano visibili in tutto il codice attuale, anche dopo la migrazione: la collection `likes` separata (non un array dentro il post) perché json-server-auth rifiutava la scrittura di un campo su una risorsa di cui non si è proprietari — un "mi piace" di un altro utente avrebbe modificato il post altrui e sarebbe stato respinto con 403 (con Firestore lo stesso schema resta comodo per gli id deterministici, vedi passaggio 12); e `App.jsx` come radice logica dell'app (Redux Provider + BrowserRouter vivono lì, oltre agli hook realtime), con `main.jsx` ridotto a puro bootstrap tecnico — separazione voluta esplicitamente, mai spostata.

Restyling grafico ispirato a LinkedIn ("inClone")

Rebranding di navbar e pagine di autenticazione, layout del feed a tre colonne (mini-profilo sticky + feed + card "Novità"), avatar di fallback con gradiente deterministico (basato sull'id utente) al posto di un cerchio blu piatto, banner di copertina sul profilo. Aggiunti campi minori: `jobTitle` (titolo professionale opzionale), split Nome/Cognome nel form di registrazione (uniti in `fullName` prima dell'invio), supporto immagine nei post.

Upload reale per l'avatar profilo e i media dei post

`ProfileEditForm` ha sostituito il campo URL testuale con un `<input type="file">`: il file scelto veniva letto con `FileReader.readAsDataURL` e la stringa risultante (base64) salvata direttamente come `avatarUrl` (limite 2MB lato client). Stessa tecnica poi estesa al campo immagine/video dei post (limiti 3MB foto / 50MB video). Questa scelta — niente object storage esterno, tutto dentro `db.json` — ha avuto una conseguenza tecnica diretta sulla fase successiva.

Bootstrap Express custom per superare il limite di json-server

Un video da 50MB codificato in base64 pesa circa 68MB, ma sia json-server che json-server-auth avevano un proprio body-parser con limite hardcoded a 10MB, non esposto da nessun flag CLI. Soluzione: sostituire l'avvio via CLI con un bootstrap Express scritto a mano (poi ulteriormente esteso per Socket.IO, rimosso del tutto con la migrazione — vedi passaggio 12).

5. Passaggi fatti nelle sessioni di lavoro

Numerazione continua e cronologica: i passaggi 1–11 sono precedenti alla migrazione a Firebase (backend JSON Server + Socket.IO, storia). I passaggi 12–14 raccontano la migrazione e il lavoro successivo, fino alla sessione odierna.

1 — Lettura del checkpoint e risoluzione di un merge in sospeso

Letto CHECKPOINT.md per il contesto del progetto. Un `git status` ha rivelato un merge Git non concluso con due file nuovi non ancora in staging: `server/db.seed.json` e `src/hooks/useInterval.js`. Un controllo con `git grep` per marcatori `<<<<<<` non aveva dato risultati — ma il file era ancora untracked, e `git grep` di default ignora i file non tracciati. Aggiungendolo esplicitamente è emerso che conteneva ancora marcatori di conflitto reali nel mezzo della collection `messages`: due messaggi reali con lo stesso id duplicato. Risolti a mano conservando entrambi i messaggi e riassegnando id univoci, poi il merge è stato concluso con un commit.

2 — Analisi architetturale approfondita e piano approvato

Per implementare 4 funzionalità grandi (messaggistica realtime, notifiche, ricerca+paginazione, modifica commenti), un agente di esplorazione dedicato ha mappato l'intera architettura, poi sono stati letti direttamente i file critici e ispezionati i sorgenti di `json-server/json-server-auth` in `node_modules` per verificare due dettagli tecnici: il secret JWT usato per firmare i token (riusabile per autenticare i socket) e il meccanismo ufficiale `router.render` per intercettare le risposte. Poste esplicitamente due domande architetturali all'utente (TypeScript? no, restare in JavaScript puro — socket.io o ws grezzo? socket.io, per riconnessione automatica). Piano scritto su file ed esplicitamente approvato prima di toccare codice.

3 — Backend — Messaggistica realtime e notifiche (era JSON Server)

Nuovo `server/realtime.js`: `authenticateSocket` verificava il JWT nell'handshake col secret di `json-server-auth`; `attachRealtime` sovrascriveva `router.render` per emettere eventi (`message:new / updated / deleted`, `conversation:new`) e creare notifiche dopo ogni scrittura. Nuova collection `notifications` con permesso 640.

 **Bug reale:** crash cascade-delete di json-server — `DELETE /comments/:id` rispondeva 500 perché json-server scansionava ogni campo `*Id` per rimuovere risorse orfane e andava in crash su un valore `null` (le notifiche di tipo messaggio non hanno `postId`). Risolto omettendo la chiave invece di impostarla a `null`.

 **Bug reale:** condizione di corsa sull'invio messaggi — il server emetteva `message:new` anche al mittente; se l'evento arrivava prima della risposta HTTP della sua stessa POST, il messaggio veniva duplicato in Redux. Risolto con un controllo "esiste già questo id?" in entrambi i reducer coinvolti.

4 — Frontend — Le 4 funzionalità (era JSON Server)

Messaggistica realtime via `socket.io-client` singleton; notifiche in tempo reale con `NotificationBell` in navbar; ricerca full-text (`?q=` di json-server) con paginazione estesa anche a commenti e messaggi (per i messaggi, ordine invertito lato client e ripristino manuale dell'offset di scroll per "carica precedenti"); modifica commenti realizzata come replica 1:1 del pattern già collaudato per la modifica messaggi.

5 — Verifica end-to-end (era JSON Server)

Senza automazione browser disponibile in quell'ambiente, verifica con script Node dedicati (chiamate HTTP e socket reali contro un backend isolato su porta di test): login di due utenti, rifiuto di un socket con token non valido, invio/modifica/eliminazione messaggi con verifica dell'evento ricevuto dall'altro utente, notifiche su commento/like con verifica di persistenza e "segna come letta", ricerca e paginazione, tentativo di modifica di un commento altrui (403 confermato). `npm run lint / build` puliti dopo ogni blocco.

6 — Documentazione aggiornata

CHECKPOINT.md e README.md aggiornati con le nuove decisioni tecniche, i limiti noti rivisti e changelog datato.

7 — Diagnosi "il sito non funziona" (era JSON Server)

Il backend in esecuzione era rimasto sul codice vecchio (Node non fa hot-reload del server). Confermato interrogando `/socket.io/?EIO=4&transport=polling` (404 → nessun socket.io agganciato), risolto riavviando. Un login segnalato come fallito è risultato un'email diversa dallo username, non un bug.

8 — Miglioramenti estetici (prima tornata)

Tema scuro (`data-bs-theme` nativo Bootstrap 5.3, script inline anti-flash in `index.html`), toast di conferma a evento globale, skeleton loader animati, NotificationBell con animazione "bump", EmptyState con icona in cerchio, micro-interazioni (pop sul like, transizioni 150-200ms su bottoni/card/link).

9 — Nuove funzionalità: seguire utenti e stato online (era JSON Server)

Nuova collection `follows`, stesso schema collaudato per `likes`. Stato online con una `Map` "userId → connessioni attive" lato server (non un booleano semplice, per gestire più schede/dispositivi), `presence:list` inviato ad ogni nuova connessione. Componente `Avatar` esteso con prop `online` opzionale.

10 — Modal di conferma e ulteriori miglioramenti grafici

I `window.confirm()` nativi sostituiti da un modale Bootstrap con lo stesso pattern "evento globale disaccoppiato" già usato per i toast (`src/utils/confirm.js`, `Promise` nascosta dentro un `CustomEvent`). Animazioni di ingresso per liste, barra di caricamento globale, dissolvenza progressiva per immagini, pagina 404 riprogettata, scrollbar personalizzata.

11 — Quattro nuove funzionalità + tre miglioramenti grafici (era JSON Server)

Feed filtrato "Chi segui" (parametro `?userId=` ripetuto, filtro OR di json-server); ricevute di lettura visuali (spunta singola/doppia stile WhatsApp); badge messaggi non letti in navbar (conteggio iniziale per conversazione, poi mantenuto solo via eventi socket); liste follower/seguiti in modale; popover di mini-profilo al passaggio del mouse (`UserHoverCard`); lightbox sulle immagini dei post; copertina di profilo personalizzabile (stesso meccanismo base64 dell'avatar).

12 — Migrazione completa a Firebase

Migrazione completa del backend da JSON Server + JSON Server Auth + Socket.IO a **Firebase** (Authentication + Firestore + Storage), su richiesta esplicita ("deve essere in tempo reale"). Il cambio architetturale più grande del progetto dopo l'implementazione iniziale: tocca praticamente ogni service/slice/hook che parla col backend.

- **Auth:** `authService` / `authSlice` riscritti su Firebase Auth (`createUserWithEmailAndPassword` / `signInWithEmailAndPassword` / `onAuthStateChanged`); il profilo esteso vive in `users/{'uid'}` su Firestore, non nel token. Rimossi `src/api/httpClient.js` e `src/utils/jwt.js`.
- **Dati:** ogni `api/*Service.js` riscritto per parlare direttamente con l'SDK Firestore. Id deterministici (`postId_userId` per i like, `followerId_followingId` per i follow) al posto degli id auto-incrementali. Paginazione passata a cursori (`where(createdAt, <, cursore)`). Il conteggio commenti su `PostCard` usa `getCountFromServer` (aggregazione lato server), dato che con la paginazione a cursore `items.length` non riflette più il totale reale.
- **Ricerca:** emulata con prefix-match su campi `*Lower` dedicati (`usernameLower` / `fullNameLower` / `contentLower`) — trova solo risultati che *iniziano* per il testo cercato. "Carica altri risultati" rifà la query con un limite più alto invece di un vero cursore. Limite noto, documentato in README.
- **Realtime:** sostituito l'intero socket.io con listener `onSnapshot` di Firestore — `useActivityRealtime` (post/commenti/like/follow, senza filtro: le Security Rules su queste collection non dipendono da altri documenti), `useConversationsRealtime` (filtrate per `participantIds` array-contains uid), `useNotificationsRealtime`

(filtrate per `userId`). I reducer `*Received` / `*UpdatedFromSocket` / `*DeletedFromSocket` sono stati riusati così come sono (stesso dedup-by-id di prima), solo la sorgente degli eventi è cambiata.

- **Scoperta chiave:** una security rule che fa `get()` su un altro documento blocca le query `"list"/ onSnapshot` senza filtro su quel campo. Fix: `participantIds` copiato dalla conversazione su ogni messaggio al momento dell'invio, così sia la regola sia la query client verificano l'appartenenza senza un `get()` incrociato.
- **Notifiche:** create client-side da chi compie l'azione (scelta esplicita per restare sul piano gratuito Spark, niente Cloud Functions). Le Security Rules impediscono di creare una notifica per sé stessi o spacciandosi per un `actorId` diverso dal proprio uid, ma non c'è validazione server-side reale.
- **Autorizzazione:** da permessi 640/660 a livello di intera collection a Firestore Security Rules (`firestore.rules`), che verificano ownership su più campi per documento — risolve davvero il vecchio limite "autorizzazione su messages/conversations solo lato client".
- **Upload:** avatar/copertina/foto dei post passati da base64-in-JSON a Firebase Storage — poi, con un follow-up nella stessa giornata (Firebase richiede il piano Blaze anche solo per abilitare Storage dal 3 febbraio 2026), **tornati a base64 diretto nei documenti Firestore** per restare gratis senza carta di credito, con limiti ridotti per stare dentro il tetto di 1MB per documento (300KB avatar/copertina, 700KB foto post). **Solo i video restano su Firebase Storage** (`storageService.uploadPostMedia`): un video da 50MB non potrebbe mai stare in un documento Firestore.
- **Rimosso:** l'intera cartella `server/` (`json-server`, `json-server-auth`, `express`, `socket.io`). I dati demo sorgente sono ora in `scripts/demoData.json`, con uno script di migrazione una tantum `scripts/seedFirestore.js` (Firebase Admin SDK) che crea gli utenti demo in Firebase Auth (stessa password `Password123!` per tutti) e popola Firestore rimappando gli id numerici del vecchio json-server sui nuovi id.
- **Id ora stringhe, non numeri:** due punti che facevano `Number(id)` sui parametri di route sono stati corretti — un residuo dell'assunzione "id numerico" di json-server che con gli uid/id-documento di Firebase (stringhe) avrebbe rotto silenziosamente i confronti `===`.
- **package.json:** rimossi `express`, `json-server`, `json-server-auth`, `jsonwebtoken`, `socket.io` / `socket.io-client`, `concurrently`. Aggiunti `firebase` (client SDK) e `firebase-admin` (solo per lo script di seed, devDependency). `axios` è rimasto: unico consumer rimasto è `newsService.js`, indipendente da questa migrazione. Script `server / dev:all` rimossi, nuovo script `seed`.

⚠ **Limite dichiarato all'utente:** nessun progetto Firebase reale disponibile per testare end-to-end in quella sessione (login, realtime multi-utente, upload, Security Rules) — solo lint e build.

13 — Prima verifica reale contro Firebase + rifiniture (stessa giornata)

Prima verifica end-to-end reale contro il progetto Firebase dell'utente, con Playwright headless (registrazione, post, like, commenti, profilo, ricerca, messaggistica tra due utenti). Trovati e corretti 2 bug reali introdotti dalla migrazione, non emersi da lint/build:

🐛 **Bug reale:** ricerca utenti sempre vuota — `authService.register()` non impostava `usernameLower` / `fullNameLower` sul documento utente; Firestore esclude dai risultati i documenti dove il campo non esiste affatto, quindi ogni utente appena registrato era invisibile alla ricerca finché non modificava il profilo. Fix: calcolati anche in `register()`.

🐛 **Bug reale:** messaggistica bloccata da `permission-denied` su tutte le conversazioni — la regola `conversations` controllava `participant1Id` / `participant2Id`, ma le query client filtravano su `participantIds` (l'array). Per una query `"list"`, Firestore nega l'intera richiesta se il campo controllato dalla regola non coincide con quello del filtro. Stesso mismatch trovato per analogia in `fetchMessages` / `fetchUnreadCount` (solo `conversationId`, non `participantIds`) prima che causasse un bug visibile. Fix: regola e query allineate su `participantIds` ovunque.

Lezione per il futuro: quando una security rule Firestore referencia un campo di `resource.data`, ogni query `"list"/onSnapshot` su quella collection deve includere un filtro esplicito sullo stesso campo, anche se logicamente ridondante con altri filtri già presenti.

Nella stessa giornata, altre rifiniture nell'era Firebase: reazioni stile LinkedIn (like/love/...) al posto del semplice "mi piace", estese anche ai commenti; realtime esteso a post/commenti/like/follow (`useActivityRealtime`, prima il realtime copriva solo messaggi/notifiche); post salvati + card "Collegamenti" in home; navbar riordinata stile LinkedIn (campo di ricerca spostato); fix posizione toast.

 **Bug reale e lezione permanente:** il menu a tendina "Condividi" (`ShareMenu`) di un post spariva/veniva coperto dal post successivo. Causa: ogni `PostCard` aveva un'animazione di ingresso (`.animate-fade-in`, dissolvenza + scorrimento) che lasciava un `transform` applicato a fine animazione — un `transform` diverso da `none` crea un nuovo stacking context, e il menu del post N finiva coperto dal Card del post N+1 (più avanti nel DOM, stesso z-index). Dopo alcuni tentativi parziali (rimuovere `fill-mode: both`, `strategy: 'fixed'` su `Popper`) il problema persisteva, quindi su richiesta esplicita l'animazione è stata rimossa del tutto: `.animate-fade-in / .empty-state-fade` restano `animation: none` in `index.css`, no-op sugli 8 componenti che le usano (`PostCard`, `CommentItem`, `JobCard`, `ConversationList`, `MessageBubble`, `NotificationBell`, `UserResultItem`, `NotFoundPage`). **Vincolo per il futuro:** non reintrodurre animazioni con `transform` persistente su questi componenti.

14 — Rifiniture UX mobile e animazioni generali (23 luglio 2026)

Giro di rifiniture mirate all'esperienza mobile della navbar e della messaggistica, seguito da un set di animazioni generali. Tutto in `AppNavbar.jsx` salvo dove indicato:

- **Menu hamburger:** contenuti centrati (`text-center text-lg-start` sul `Nav`, allineati a sinistra solo da `lg` in su); blocco avatar+nome profilo spostato fuori dal `Nav`, centrato in cima al menu (avatar a sinistra del nome).
- **Barra di ricerca:** prima era `d-none d-sm-block`, quindi **invisibile sotto i 576px senza alcuna alternativa** nel menu hamburger. Ora è sempre visibile nella barra principale (flessibile, `maxWidth: 280`), fuori dall'hamburger.
- **Tema e notifiche:** spostati fuori dall'hamburger, sempre visibili. Due istanze responsive distinte: una `d-lg-none` nella barra principale (mobile), una `d-none d-lg-flex` dentro il `Nav` tra "Lavoro" e l'avatar (desktop, così appaiono immediatamente a sinistra dell'avatar). `NotificationBell` ha ricevuto una prop `id` configurabile per evitare id DOM duplicati con due istanze montate.
- **FeedPage.jsx**: link "Elementi salvati" aggiunto accanto a "Chi segui" nella Nav pills (visibile solo `d-lg-none`), dato che prima viveva solo nella sidebar sinistra nascosta sotto `lg` — invisibile su mobile.
- **ConversationView.jsx**: freccia "indietro" verso `/messages`, visibile solo mobile (`d-md-none`) — prima non c'era modo di tornare alla lista conversazioni senza il tasto indietro del browser.
- **MessengerWidget.jsx**: il pannello di conversazione apriva un secondo box fluttuante affiancato al box della lista (`right: 320`) che su schermi stretti usciva dal bordo sinistro dello schermo, tagliato. Consolidato in un solo pannello (la conversazione sostituisce la lista nello stesso box), larghezza `min(300px, calc(100vw - 1.5rem))`.
- **Animazioni generali** aggiunte in `index.css` — solo opacity/scale su elementi foglia, coerentemente col vincolo del passaggio 13 (niente `transform` su contenitori con dropdown `Popper` dentro): `.page-transition` (dissolvenza pagina su cambio rotta, key su `location.pathname` in `MainLayout.jsx`); transizione slide+dissolvenza su `.toast`; `.badge-pop` (bump sul badge non letti di `NotificationBell`, preserva il `translate-middle` esistente) e lo stesso `.like-pop` riusato sul badge "Messaggi"; burst "+1" fluttuante (`.reaction-burst`) su `ReactionButton.jsx` al click su una reazione.

 **Limite dichiarato all'utente:** nessuno strumento di automazione browser disponibile in questa sessione — verificato solo con lint e build, non in un browser reale.

6. File principali toccati

Area	File nuovi	File modificati (principali)
Backend realtime (era JSON Server)	server/realtime.js	server/server.js, server/db.json, server/routes.json
Frontend realtime/notifiche (era JSON Server)	src/socket.js, src/hooks/useConversationSocket.js, src/features/notifications/*, src/components/notifications/*	src/App.jsx, src/app/store.js, src/features/messages/messagesSlice.js, src/components/messages/*
Ricerca & paginazione (era JSON Server)	src/features/search/*, src/pages/SearchPage.jsx, src/components/search/*, src/hooks/useDebounce.js	src/api/usersService.js, src/api/postsService.js, src/features/posts/postsSlice.js
Commenti	—	src/api/commentsService.js, src/features/comments/commentsSlice.js, src/components/comments/*
Estetica (1ª tornata)	src/hooks/useTheme.js, src/utils/toast.js, src/components/common/ToastHost.jsx, src/app/toastMiddleware.js, skeleton*	src/index.css, index.html, src/components/layout/AppNavbar.jsx, src/components/common/EmptyState.jsx, src/components/posts/PostCard.jsx
Seguire utenti / stato online (era JSON Server)	src/features/follow/followSlice.js, src/features/presence/presenceSlice.js	server/realtime.js, src/App.jsx, src/components/common/Avatar.jsx, src/components/profile/ProfileCard.jsx
Modal di conferma	src/utils/confirm.js, src/components/common/ConfirmModalHost.jsx	src/App.jsx, src/components/posts/PostCard.jsx, src/components/messages/ConversationView.jsx
Feed "Chi segui" / ricevute lettura / badge non letti (era JSON Server)	src/components/posts/FollowingFeedList.jsx	src/api/postsService.js, src/pages/FeedPage.jsx, src/components/messages/MessageBubble.jsx, src/components/layout/AppNavbar.jsx
Popover / lightbox / copertina profilo	src/components/common/UserHoverCard.jsx	src/components/posts/PostCard.jsx, src/components/profile/ProfileEditForm.jsx
Migrazione Firebase	src/firebase.js, firestore.rules, storage.rules, firebase.json, firestore.indexes.json, scripts/demoData.json, scripts/seedFirestore.js, src/hooks/useActivityRealtime.js, src/hooks/useConversationsRealtime.js, src/hooks/useNotificationsRealtime.js, src/hooks/usePresence.js, src/api/storageService.js	ogni src/api/*Service.js, ogni *Slice.js con paginazione, src/App.jsx, package.json (rimossa cartella server/intera)
Rifiniture era Firebase (reazioni, realtime esteso, post salvati)	—	src/components/common/IconButton.jsx, src/utils/reactions.js, src/hooks/useActivityRealtime.js, src/pages/SavedPostsPage.jsx, src/components/layout/AppNavbar.jsx, src/index.css
Rifiniture UX mobile e animazioni (23 luglio)	—	src/components/layout/AppNavbar.jsx, src/components/layout/MainLayout.jsx, src/pages/FeedPage.jsx, src/components/messages/ConversationView.jsx, src/components/messages/MessengerWidget.jsx, src/components/common/IconButton.jsx, src/components/notifications/NotificationBell.jsx, src/index.css
Documentazione	—	CHECKPOINT.md, README.md, documentazione-progetto.pdf (questo file)

✓ Stato attuale

`npm run lint` e `npm run build` puliti dopo ogni fase, incluse tutte le modifiche di oggi. La migrazione a Firebase è stata verificata end-to-end con Playwright headless contro un progetto Firebase reale (registrazione, post, like, commenti, profilo, ricerca, messaggistica tra due utenti), con due bug reali trovati e corretti (vedi passaggio 13). Le rifiniture UX mobile e le animazioni generali del passaggio 14 **non sono ancora state verificate in un browser reale** — nessuno strumento di automazione browser disponibile in questa sessione, dichiarato esplicitamente invece di dare il lavoro per "verificato al 100%". Working tree sul branch `Lucho` : la maggior parte del lavoro descritto in questo documento è già stata committata nel corso delle sessioni; da rivedere e completare il commit delle modifiche restanti quando l'utente lo ritiene opportuno.

Documento generato/aggiornato a partire dalla cronologia delle sessioni di lavoro con Claude Code.